

Single-Store Multi-Vendor

: 1.0 | : 15.07.2026

1.

1.1.

- () , :
- **Single-Store** — -
 - **Multi-Vendor** —

1.2.

single-store multi-vendor , **100 000+** , .

1.3.

- B2C B2B
- (handmade, ,)
-
- White-label

1.4.

1	, DevOps,	6	
2	Core API (Auth, Users, Catalog)	8	
3	E-commerce (Cart, Orders, Payments)	8	
4	Multi-vendor (Vendors, Commissions, Payouts)	6	
5	Frontend (Storefront, Dashboards)	10	
6	Search, Analytics, Real-time	6	
7	, ,	6	
		50	(~12)

2.

2.1.

2.1.1. (Single-Store / Multi-Vendor) FR-1.1.

single-store multi-vendor tenant' .

FR-1.2. single-store:

- (tenant)
- vendor'
- UI (,)
-

FR-1.3. multi-vendor:

- vendor'
- vendor-
- ()
- Split- (vendor')
- vendor'

FR-1.4. tenant' - .

2.1.2. (Auth Service) FR-2.1. :

- Email + (bcrypt, min 8)
- OAuth 2.0 (Google, Apple, Facebook)
- OTP (SMS / Email) 2FA
- JWT access token (15) + refresh token (7)

FR-2.2. (RBAC):

super_admin	tenant'
tenant_admin	tenant'
vendor	
customer	
manager	tenant' (,)
moderator	

FR-2.3. Row Level Security (RLS) — tenant_id.

FR-2.4. Rate limiting: 100 / , 1000/ .

2.1.3. (Catalog Service) FR-3.1. (,).

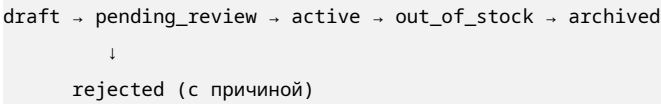
FR-3.2. :

- (, ,)
- (SKU-level: + = SKU)
-
-

FR-3.3. :

- 20
- (100MB)
- 3D- (GLB/GLTF)
- CDN- (WebP,)

FR-3.4. :



FR-3.5. Bulk- :

- CSV/Excel (10 000)
-
- Bulk edit (, ,)

FR-3.6. SEO:

- URL (slug)
- Meta title, description, keywords
- Open Graph tags
- Schema.org JSON-LD
- Sitemap.xml ()

2.1.4. (Search Service) FR-4.1. :

- (<50ms)
- Typo-tolerance (2)
- ()
- ()
- (, , ,)

FR-4.2. :

-
-

- CTR

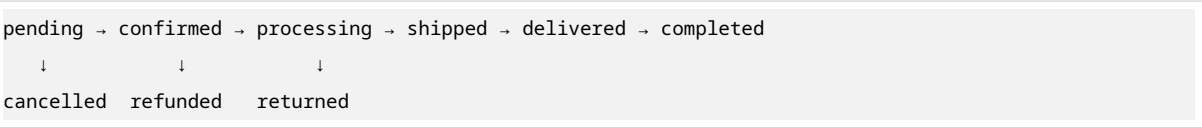
2.1.5. (Cart & Checkout Service) FR-5.1. :

- (localStorage + merge)
- -vendor ()
-
-
-

FR-5.2. ():

-
- ()
- ()
-
-
- ()

FR-5.3. :



2.1.6. (Payments Service) FR-6.1. :

- **Stripe** () — , Apple Pay, Google Pay
- **Stripe Connect** (multi-vendor) —
- **PayPal**
- (manual confirmation)
- (Bitcoin, Ethereum —)

FR-6.2. Single-store:

- tenant'

FR-6.3. Multi-vendor:

- Split-
- (%)
- vendor' (/)
- (escrow) —
-

FR-6.4. :

- PCI DSS compliance (Stripe —)
- 3D Secure
- Idempotency keys
- Webhook verification

2.1.7. Multi-Vendor (Vendor Service) FR-7.1. vendor' :

- (KYC: ,)
-
- Stripe Connect account

FR-7.2. Vendor Dashboard:

- (,)
-
-
-
- (, ,)
- (, ,)

FR-7.3. :

- (%)
- vendor'
-
- (tiered)

FR-7.4. Vendor- :

- vendor' (, ,)
- vendor'
- vendor'
- (,)

2.1.8. (Reviews Service) FR-8.1. :

- (verified purchase)
- 1-5
- + /
- ()
- vendor'

FR-8.2. vendor' (multi-vendor):

-

- : , ,

FR-8.3. :

- (AI: ,)
- (moderator queue)

2.1.9. (Notifications Service) FR-9.1. :

- Email (SendGrid / AWS SES)
- Push (Firebase Cloud Messaging)
- SMS (Twilio)
- In-app notifications
- WebSocket real-time

FR-9.2. :

- / / /
-
-
- -
-

FR-9.3. :

-
- (, ,)

2.1.10. (Analytics Service) FR-10.1. tenant' :

- , ,
-
- vendor' (multi-vendor)
-
-

FR-10.2. vendor' :

- , ,
-
-

FR-10.3. Real-time :

-
-
- Revenue per minute

2.2.

2.2.1.

API (p95)	< 100ms	, < 300ms
(LCP)	< 2.5s	
First Input Delay (FID)	< 100ms	
Cumulative Layout Shift (CLS)	< 0.1	
	100 000+ concurrent users	
	< 50ms	
(submit to confirmation)	< 3s	

2.2.2.

Uptime SLA	99.99%	
RTO (Recovery Time Objective)	< 15	
RPO (Recovery Point Objective)	< 1	
	Graceful degradation (	, read-only)

2.2.3.

	-	CPU/memory
		tenant_id (Citrus)
		Redis Cluster,
		Elasticsearch cluster, 3+
		MinIO/S3 CDN

2.2.4.

- OWASP Top 10 compliance
- SQL Injection prevention (parameterized queries)
- XSS protection (CSP headers, output encoding)
- CSRF tokens
- Rate limiting (per IP, per user, per endpoint)
- Data encryption at rest (AES-256) in transit (TLS 1.3)
- GDPR / CCPA compliance (data export, deletion, consent)
- Audit log ()
- Penetration testing ()

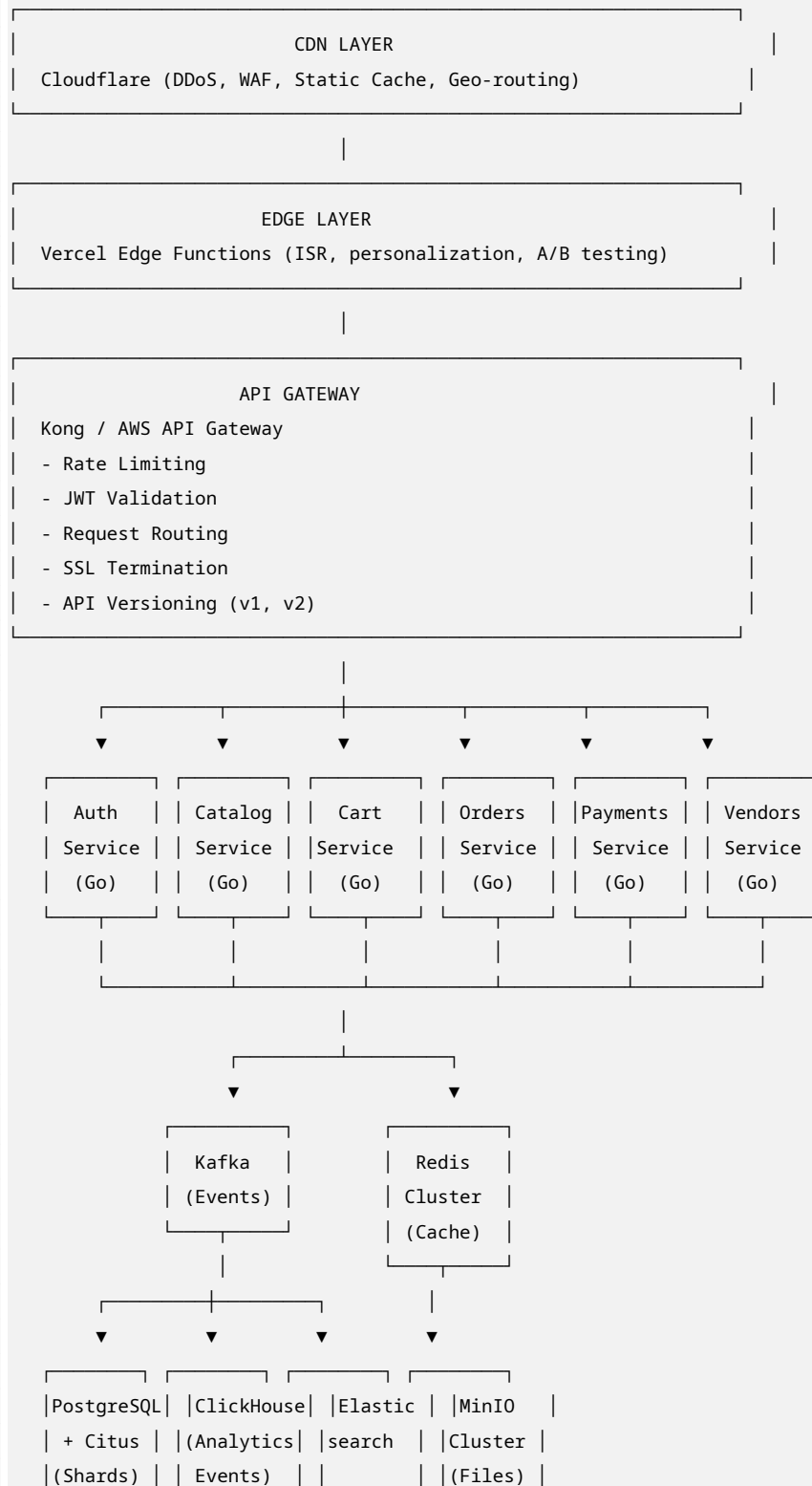
2.2.5.

- 20+
- RTL (Arabic, Hebrew)
- (150+ ,)

- (VAT, sales tax)
- / /

3.

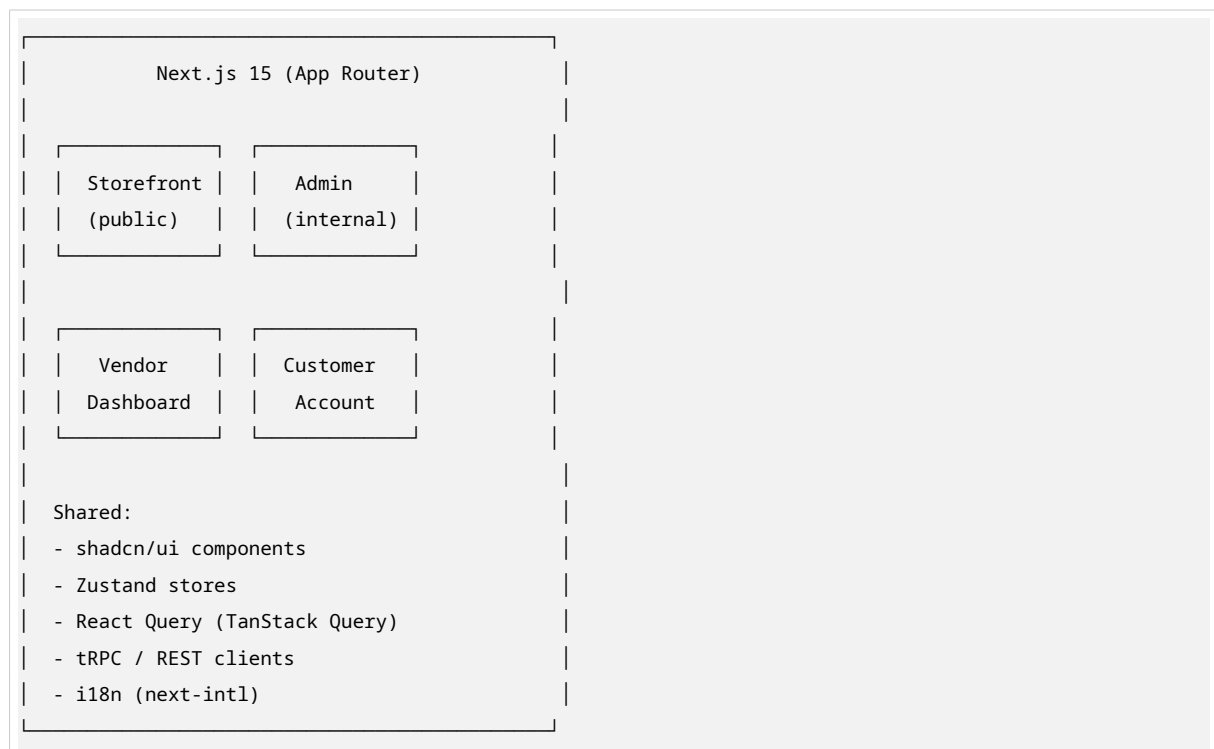
3.1.



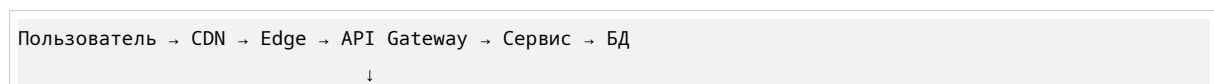
3.2.

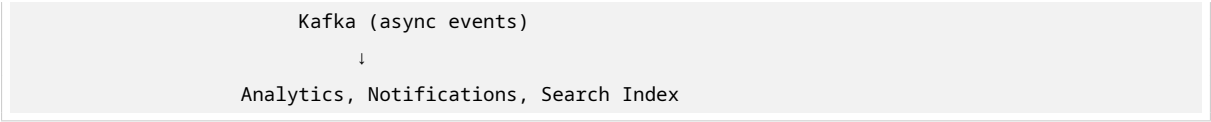
API Gateway	Go (Kong)	—	, rate limiting, auth
Auth Service	Go	PostgreSQL	, , JWT, RBAC, sessions
Catalog Service	Go	PostgreSQL + Redis	, , ,
Search Service	Go	Elasticsearch	, ,
Cart Service	Go	Redis + PostgreSQL	, ,
Orders Service	Go	PostgreSQL	, ,
Payments Service	Go	PostgreSQL	, refunds, split-payments, payouts
Vendor Service	Go	PostgreSQL	Vendor , ,
Reviews Service	Go	PostgreSQL	, ,
Notifications Service	Go	PostgreSQL + Redis	Email, push, SMS, in-app
Analytics Service	Go	ClickHouse	, , real-time
Media Service	Go	MinIO	, , CDN-
Real-time Service	Go (Centrifugo)	Redis	WebSockets, SSE,

3.3. Frontend



3.4. Data Flow





3.5. Event-Driven Architecture (Kafka Topics)

Topic		
user.registered	Auth	Notifications, Analytics
product.created	Catalog	Search (index), Analytics
product.updated	Catalog	Search (reindex), Cache (invalidate)
cart.updated	Cart	Analytics, Recommendations
order.created	Orders	Payments, Notifications, Inventory
order.paid	Payments	Orders, Notifications, Vendor (payout)
order.shipped	Orders	Notifications, Tracking
review.submitted	Reviews	Notifications, Vendor (rating update)
vendor.registered	Vendor	Auth, Payments (Stripe Connect), Notifications

4.

4.1. Frontend

Framework	Next.js	15.x
Language	TypeScript	5.8.x
Runtime	React	19.x
Routing	App Router (Server Components)	—
Styling	Tailwind CSS	4.x
Components	shadcn/ui + Radix UI	latest
State (client)	Zustand	5.x
State (server)	TanStack Query (React Query)	5.x
Forms	React Hook Form + Zod	latest
i18n	next-intl	latest
Charts	Recharts / Tremor	latest
Maps	Leaflet / Mapbox	latest
Testing	Vitest + React Testing Library + Playwright	latest
Build	Turborepo (monorepo)	latest

4.2. Backend

Language	Go	1.24.x
Web Framework	Gin / Echo	latest
gRPC	protobuf + grpc-go	latest
ORM	GORM / sqlx	latest
Validation	go-playground/validator	latest
Auth	JWT (golang-jwt) + bcrypt	latest
Testing	testify + mockery	latest
Documentation	Swagger (swaggo)	latest

4.3.

Primary DB	PostgreSQL 16 + Citus	
Cache	Redis 7 Cluster	, , rate limiting,
Search	Elasticsearch 8	
Analytics	ClickHouse	OLAP, ,
NoSQL ()	ScyllaDB	, ,
Files	MinIO Cluster	S3-compatible, ,
Message Queue	Apache Kafka 3.x	Event streaming
Job Queue	BullMQ (Redis-based)	

4.4.

Containerization	Docker + Docker Compose (dev), Kubernetes (prod)
Orchestration	Kubernetes (EKS / GKE)
Service Mesh	Istio
API Gateway	Kong / AWS API Gateway
CDN	Cloudflare
Edge	Vercel Edge Functions
CI/CD	GitHub Actions
GitOps	ArgoCD
IaC	Terraform + Pulumi
Monitoring	Prometheus + Grafana
Logging	Loki + Grafana
Tracing	Jaeger / OpenTelemetry
APM	Datadog / New Relic
Error Tracking	Sentry
Secrets	HashiCorp Vault / AWS Secrets Manager

4.5.

Stripe	Stripe Connect	, split-payments, payouts
PayPal	REST API	
SendGrid / AWS SES	SMTP / API	Email-
Twilio	REST API	SMS
Firebase	FCM	Push-
Google Maps	Places API	,
ShipStation / EasyPost	REST API	,

5.

5.1. (Citrus)

```
-- Distributed table by tenant_id
SELECT create_distributed_table('products', 'tenant_id');
SELECT create_distributed_table('orders', 'tenant_id');
SELECT create_distributed_table('users', 'tenant_id');

-- Reference tables (replicated to all nodes)
SELECT create_reference_table('categories');
SELECT create_reference_table('attributes');
```

5.2.

tenants

```
CREATE TABLE tenants (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  slug VARCHAR(255) UNIQUE NOT NULL,
  mode VARCHAR(20) NOT NULL CHECK (mode IN ('single_store', 'multi_vendor')),
  settings JSONB DEFAULT '{}',
  commission_rate DECIMAL(5,2) DEFAULT 10.00, -- %
  status VARCHAR(20) DEFAULT 'active',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

users

```
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL,
  email VARCHAR(255) NOT NULL,
  password_hash VARCHAR(255),
  role VARCHAR(50) NOT NULL,
  first_name VARCHAR(100),
  last_name VARCHAR(100),
```

```

phone VARCHAR(20),
avatar_url VARCHAR(500),
email_verified BOOLEAN DEFAULT FALSE,
status VARCHAR(20) DEFAULT 'active',
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(tenant_id, email)
);

```

vendors (multi-vendor mode)

```

CREATE TABLE vendors (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL,
  user_id UUID NOT NULL, -- owner
  name VARCHAR(255) NOT NULL,
  slug VARCHAR(255) NOT NULL,
  description TEXT,
  logo_url VARCHAR(500),
  banner_url VARCHAR(500),
  commission_rate DECIMAL(5,2), -- override tenant rate
  stripe_account_id VARCHAR(255),
  status VARCHAR(20) DEFAULT 'pending', -- pending, active, suspended
  kyc_verified BOOLEAN DEFAULT FALSE,
  rating DECIMAL(2,1) DEFAULT 0.0,
  review_count INTEGER DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(tenant_id, slug)
);

```

products

```

CREATE TABLE products (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL,
  vendor_id UUID, -- NULL for single-store
  category_id UUID NOT NULL,
  name VARCHAR(500) NOT NULL,
  slug VARCHAR(500) NOT NULL,
  description TEXT,
  short_description VARCHAR(1000),
  sku VARCHAR(100),
  price DECIMAL(12,2) NOT NULL,
  compare_at_price DECIMAL(12,2),
  cost_price DECIMAL(12,2),
  currency VARCHAR(3) DEFAULT 'USD',
  inventory_quantity INTEGER DEFAULT 0,
  inventory_policy VARCHAR(20) DEFAULT 'deny', -- deny, continue
  weight DECIMAL(10,2),
  weight_unit VARCHAR(10) DEFAULT 'kg',

```

```

status VARCHAR(20) DEFAULT 'draft',
is_featured BOOLEAN DEFAULT FALSE,
seo_title VARCHAR(70),
seo_description VARCHAR(320),
seo_keywords VARCHAR(500),
attributes JSONB DEFAULT '{}',
metadata JSONB DEFAULT '{}',
rating DECIMAL(2,1) DEFAULT 0.0,
review_count INTEGER DEFAULT 0,
sales_count INTEGER DEFAULT 0,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(tenant_id, slug)
);

```

product_variants

```

CREATE TABLE product_variants (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  product_id UUID NOT NULL,
  sku VARCHAR(100) NOT NULL,
  price DECIMAL(12,2),
  compare_at_price DECIMAL(12,2),
  inventory_quantity INTEGER DEFAULT 0,
  options JSONB NOT NULL, -- {"color": "red", "size": "M"}
  image_url VARCHAR(500),
  is_default BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

```

orders

```

CREATE TABLE orders (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL,
  user_id UUID NOT NULL,
  order_number VARCHAR(50) NOT NULL,
  status VARCHAR(20) DEFAULT 'pending',
  payment_status VARCHAR(20) DEFAULT 'pending',
  fulfillment_status VARCHAR(20) DEFAULT 'unfulfilled',
  currency VARCHAR(3) DEFAULT 'USD',
  subtotal DECIMAL(12,2) NOT NULL,
  discount_total DECIMAL(12,2) DEFAULT 0,
  shipping_total DECIMAL(12,2) DEFAULT 0,
  tax_total DECIMAL(12,2) DEFAULT 0,
  total DECIMAL(12,2) NOT NULL,
  shipping_address JSONB NOT NULL,
  billing_address JSONB,
  notes TEXT,
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW(),
);

```

```
    updated_at TIMESTAMPTZ DEFAULT NOW(),
    UNIQUE(tenant_id, order_number)
);
```

order_items

```
CREATE TABLE order_items (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    order_id UUID NOT NULL,
    product_id UUID NOT NULL,
    variant_id UUID,
    vendor_id UUID, -- for multi-vendor split
    name VARCHAR(500) NOT NULL,
    sku VARCHAR(100),
    quantity INTEGER NOT NULL,
    unit_price DECIMAL(12,2) NOT NULL,
    total_price DECIMAL(12,2) NOT NULL,
    commission_amount DECIMAL(12,2) DEFAULT 0,
    vendor_payout_amount DECIMAL(12,2) DEFAULT 0,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
```

payments

```
CREATE TABLE payments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    order_id UUID NOT NULL,
    user_id UUID NOT NULL,
    amount DECIMAL(12,2) NOT NULL,
    currency VARCHAR(3) DEFAULT 'USD',
    provider VARCHAR(50) NOT NULL, -- stripe, paypal
    provider_payment_id VARCHAR(255),
    status VARCHAR(20) DEFAULT 'pending',
    payment_method VARCHAR(50),
    metadata JSONB DEFAULT '{}',
    created_at TIMESTAMPTZ DEFAULT NOW()
);
```

vendor_payouts

```
CREATE TABLE vendor_payouts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    vendor_id UUID NOT NULL,
    amount DECIMAL(12,2) NOT NULL,
    commission_total DECIMAL(12,2) NOT NULL,
    status VARCHAR(20) DEFAULT 'pending', -- pending, processing, completed, failed
    stripe_transfer_id VARCHAR(255),
    period_start DATE NOT NULL,
```

```
period_end DATE NOT NULL,  
created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

5.3.

```
-- Поисквые  
CREATE INDEX idx_products_tenant_status ON products(tenant_id, status);  
CREATE INDEX idx_products_vendor ON products(vendor_id) WHERE vendor_id IS NOT NULL;  
CREATE INDEX idx_products_category ON products(category_id);  
CREATE INDEX idx_products_featured ON products(tenant_id, is_featured) WHERE is_featured = TRUE;  
  
-- Полнотекстовый поиск  
CREATE INDEX idx_products_search ON products USING gin(to_tsvector('english', name || ' ' ||  
↪ COALESCE(description, '')));  
  
-- Заказы  
CREATE INDEX idx_orders_user ON orders(user_id);  
CREATE INDEX idx_orders_status ON orders(tenant_id, status);  
CREATE INDEX idx_orders_created ON orders(created_at);  
  
-- RLS Policies  
ALTER TABLE products ENABLE ROW LEVEL SECURITY;  
CREATE POLICY tenant_isolation ON products  
    USING (tenant_id = current_setting('app.current_tenant')::UUID);
```

6. API

6.1. REST API ()

URL: https://api.{tenant}.marketplace.com/v1

Auth

```
POST /v1/auth/register  
POST /v1/auth/login  
POST /v1/auth/refresh  
POST /v1/auth/logout  
POST /v1/auth/forgot-password  
POST /v1/auth/reset-password  
POST /v1/auth/oauth/{provider}  
GET /v1/auth/me  
PUT /v1/auth/me
```

Products

GET	/v1/products	# List (pagination, filters, sort)
GET	/v1/products/{slug}	# Detail
GET	/v1/products/{slug}/related	# Related products
GET	/v1/products/{slug}/reviews	# Product reviews

POST	/v1/products	# Create (vendor/admin)
PUT	/v1/products/{id}	# Update
DELETE	/v1/products/{id}	# Delete (soft)
POST	/v1/products/{id}/images	# Upload images

Categories

GET	/v1/categories	# Tree structure
GET	/v1/categories/{slug}/products	# Products in category

Cart

GET	/v1/cart	# Get cart
POST	/v1/cart/items	# Add item
PUT	/v1/cart/items/{id}	# Update quantity
DELETE	/v1/cart/items/{id}	# Remove item
POST	/v1/cart/apply-coupon	# Apply promo code
DELETE	/v1/cart/coupon	# Remove coupon
POST	/v1/cart/shipping-estimate	# Calculate shipping

Orders

POST	/v1/orders	# Create from cart
GET	/v1/orders	# List (customer)
GET	/v1/orders/{id}	# Detail
POST	/v1/orders/{id}/cancel	# Cancel
GET	/v1/orders/{id}/tracking	# Tracking info

Payments

POST	/v1/payments/intent	# Create payment intent
POST	/v1/payments/confirm	# Confirm payment
POST	/v1/payments/webhooks/stripe	# Stripe webhook

Vendors (multi-vendor)

POST	/v1/vendors/apply	# Apply as vendor
GET	/v1/vendors	# List
GET	/v1/vendors/{slug}	# Public profile
GET	/v1/vendors/{slug}/products	# Vendor products
GET	/v1/vendors/{slug}/reviews	# Vendor reviews

Vendor Dashboard (protected)

GET	/v1/vendor/dashboard/stats	# Sales stats
GET	/v1/vendor/orders	# Vendor orders
GET	/v1/vendor/products	# Vendor products
GET	/v1/vendor/payouts	# Payout history
GET	/v1/vendor/settings	# Vendor settings
PUT	/v1/vendor/settings	# Update settings

Search

```
GET    /v1/search?q={query}&filters={}&sort={}&page={}&limit={}
GET    /v1/search/suggest?q={query}      # Autocomplete
GET    /v1/search/facets?q={query}       # Available filters
```

Reviews

```
POST   /v1/products/{id}/reviews          # Create review (verified purchase)
GET     /v1/products/{id}/reviews      # List
POST    /v1/reviews/{id}/helpful      # Mark helpful
```

6.2. gRPC API ()

```
syntax = "proto3";

package catalog;

service CatalogService {
  rpc GetProduct(GetProductRequest) returns (Product);
  rpc ListProducts(ListProductsRequest) returns (ProductList);
  rpc CreateProduct(CreateProductRequest) returns (Product);
  rpc UpdateProduct(UpdateProductRequest) returns (Product);
  rpc DeleteProduct(DeleteProductRequest) returns (Empty);
  rpc UpdateInventory(UpdateInventoryRequest) returns (InventoryResponse);
}

message Product {
  string id = 1;
  string tenant_id = 2;
  string vendor_id = 3;
  string name = 4;
  string slug = 5;
  double price = 6;
  int32 inventory_quantity = 7;
  string status = 8;
  repeated string image_urls = 9;
}
```

6.3. WebSocket Events (Real-time)

```
// Connection
const ws = new WebSocket('wss://ws.marketplace.com');
ws.send(JSON.stringify({
  type: 'subscribe',
  channels: ['orders', 'notifications'],
  token: 'JWT_TOKEN'
}));

// Events
{
```

```
"type": "order.status_updated",
"data": {
  "order_id": "...",
  "status": "shipped",
  "tracking_number": "..."
}
}
```

7. FRONTEND

7.1. Storefront

SEO		
/	, featured products,	✓
/products		✓
/products/{slug}		✓
/categories/{slug}		✓
/vendors	vendor' (multi-vendor)	✓
/vendors/{slug}	vendor'	✓
/cart		✗ (noindex)
/checkout		✗
/orders		✗
/orders/{id}		✗
/account		✗
/search?q=		✓

7.2. Admin Dashboard

Overview	KPI, , real-time
Products	CRUD, bulk, import/export
Orders	, , refunds
Customers	, ,
Vendors	, , payouts
Categories	,
Promotions	, ,
Reviews	
Settings	, SEO, ,
Analytics	,

7.3. Vendor Dashboard

Overview	, ,
----------	-----

Table 15 – continued

Products	CRUD,
Orders	
Analytics	
Finances	,
Reviews	
Settings	,

8.

8.1.

- JWT access token (RS256, 15)
- Refresh token (rotate, 7)
- Fingerprint binding (device + IP)
- Brute-force protection (5 → lock 15)

8.2.

- RBAC middleware
- Resource-level permissions (“ ”)
- RLS PostgreSQL

8.3.

- AES-256 sensitive data at rest
- TLS 1.3 in transit
- HashiCorp Vault secrets
- PCI DSS (Stripe,)

8.4. Compliance

- GDPR: data export, right to erasure, consent tracking
- CCPA: opt-out, data disclosure
- Cookie consent banner
- Privacy policy, Terms of service

9.

9.1.

Unit	Go test, Vitest	> 80%
Integration	Postman / k6	API endpoints
E2E	Playwright	user flows

Table 16 – continued

Load	k6 / Locust	100k concurrent
Security	OWASP ZAP	
Chaos	Gremlin	

9.2.

```
// k6 script example
import http from 'k6/http';
import { check } from 'k6';

export const options = {
  stages: [
    { duration: '2m', target: 10000 }, // Ramp up
    { duration: '5m', target: 100000 }, // Steady state
    { duration: '2m', target: 200000 }, // Spike
    { duration: '5m', target: 100000 }, // Recovery
    { duration: '2m', target: 0 }, // Ramp down
  ],
  thresholds: {
    http_req_duration: ['p(95)<300'],
    http_req_failed: ['rate<0.01'],
  },
};

export default function () {
  const res = http.get('https://api.marketplace.com/v1/products');
  check(res, {
    'status is 200': (r) => r.status === 200,
    'response time < 300ms': (r) => r.timings.duration < 300,
  });
}
```

10.

10.1. CI/CD Pipeline

```
Git Push → GitHub Actions → Test → Build Docker → Push Registry
      ↓
      ArgoCD (GitOps)
      ↓
      Kubernetes (Staging → Production)
```

10.2.

CPU/Memory	Prometheus	> 80%
Response Time	Prometheus	p95 > 300ms
Error Rate	Sentry	> 0.1%
DB Connections	Prometheus	> 80%
Queue Depth	Prometheus	> 1000
Disk Space	Prometheus	> 85%

10.3.

- Structured logs (JSON)
- Correlation ID (trace across services)
- Retention: 30 (hot), 1 (cold S3)

11.

11.1.

	-	
Tech Lead / Architect	1	, - ,
Backend (Go) Senior	2	Core services
Backend (Go) Middle	3	Services, integrations
Frontend (Next.js) Senior	2	Storefront, architecture
Frontend (Next.js) Middle	2	Dashboards, components
DevOps / SRE	2	Infra, CI/CD, monitoring
QA Engineer	2	Testing, automation
DBA	1	,
Product Manager	1	,
UI/UX Designer	1	,
	17	

11.2.

1. + DevOps	1-6	, CI/CD,
2. Auth + Users	7-10	, , RBAC
3. Catalog + Search	11-18	, , Elasticsearch
4. Cart + Orders	19-26	, ,
5. Payments	27-32	Stripe, split-payments
6. Multi-Vendor	33-38	Vendor' , , payouts
7. Frontend Storefront	19-38 ()	, ,
8. Frontend Dashboards	33-44	Admin, Vendor dashboards
9. Notifications +	39-44	WebSockets, email, push
Real-time		

Table 19 – continued

10. Analytics	45-48	ClickHouse,
11. +	49-52	,
	52	Production-ready
12. ()		
12.1. ()		
EKS (Kubernetes)	\$5,000	
PostgreSQL (RDS / managed)	\$3,000	
Redis Cluster	\$1,500	
Elasticsearch	\$2,000	
Kafka	\$2,000	
ClickHouse	\$1,500	
MinIO / S3	\$1,000	
CDN (Cloudflare)	\$500	
Vercel (Enterprise)	\$2,000	
Monitoring (Datadog)	\$2,000	
	~\$20,500/	
12.2. ()		
17	~\$8,000 avg	\$136,000
		~\$136,000/
12.3. (12)		
(12)	\$1,632,000	
(12)	\$246,000	
, SaaS	\$50,000	
(20%)	\$385,600	
	~\$2,313,600	
13.		

Vendor lock-in (Stripe)	payment provider
	, encryption
	Buffer time,
	MVP single-store,
multi-vendor	multi-vendor

14.

Tenant	()
Vendor	multi-vendor
SKU	Stock Keeping Unit –
RLS	Row Level Security –
CQRS	Command Query Responsibility Segregation
Saga	

- [Next.js Documentation](#)
- [Go Documentation](#)
- [Citrus Documentation](#)
- [Stripe Connect](#)
- [Apache Kafka](#)
- [Kubernetes](#)

: 15.07.2026 : 1.0 :